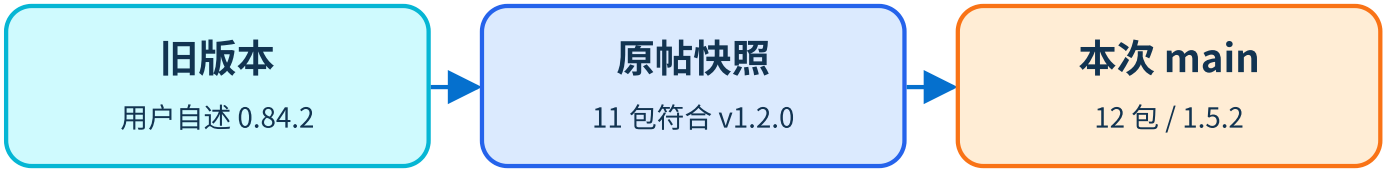
Pi 不是一个包
但你的版本只有一个

0.84.2 ≠ 1.2.0 ≠ 1.5.2

从一张依赖图，学会区分产品、库、扩展与安全边界。

不调用任何付费模型，真实验证旧版本发布物与安装遥测分支。

本次是增量学习，不是重新冒充独立审核。
大量复用既有独立核查报告（evidence/prior-audit.md），新增当前 main 查重、HEAD 漂移复核、12 个问题示意图与六步真实组件实验。



同一个 npm 产品包，不同时间切片

四类证据标签

【作者说法】X 主帖、引用与回复的观点，不等于官方保证。
【源码证实】固定提交或发布物中的静态实现/声明。
【我实际运行过（云端隔离环境）】只限日志所列六步组件检查。
【未验证】用户本机、完整 CLI、真实模型/MCP、服务端隐私行为。

原帖：AI少年 @aehyok · 2114233439205679614
发帖：2026-10-25 13:51:47 UTC+8；本次学习：2026-11-11 UTC+8。清单“久未查看”日期不是发帖日期。

没有更改用户配置、安全设置、reminder、共享 vault、Git 发布或定时任务。云端目录隔离不等于操作系统安全沙箱；本次没有启动完整 Agent。

先钉版本，再数依赖

快照	固定完整 SHA	提交时间（上海）
用户前提 / v0.84.2	`914cf1472e715297caa30db4b9535d534a9eb718`	2026-08-14 18:00:00 UTC+8
历史 v1.2.0	`06a466935536f8c9bcacb2d5efc0fc021be1bdff`	2026-10-13 06:46:55 UTC+8
复用旧审计 HEAD	`65c43bb520edf53f11923f7a927aefd05475c6af`	2026-11-10 22:56:34 UTC+8
本次 main	`b30d54b8aebe6408e02d1473446d25f8083da68f`	2026-11-11 00:38:00 UTC+8

【我实际运行过（云端隔离环境）】本次发现 main 已比旧审计前进；产品仍为 1.5.2。7 个关键文件逐字节相同，四快照的 package.json 图另行重算。旧报告所有结论仍以旧 SHA 为准，不把旧 HEAD 称为最新。

项目 / 生产依赖图口径	0.84.2	1.2.0	main 1.5.2
packages/*/package.json	9	11	12
非 private 主包	8	10	11
产品自家生产依赖闭包（含本体）	7	7	8
内置 MCP / codemode	无后来实现	有	有
直接自家依赖	agent-core / ai / client / protocol / tui	agent-core / ai / codemode / mcp / tui	

【源码证实＋我实际运行过（云端隔离环境）】计数来自四快照的主工作区清单与生产依赖闭包，不是所有 npm 外部依赖数，不含 peer/dev，不是运行时加载追踪。0.84.2 与 1.2.0 同为 7，不代表组成相同。

证据：packages/coding-agent/package.json:43-L65 · b30d54b8；packages/coding-agent/package.json:42-L83 · 914cf147。机器输出：exercise/graph-results.json。

由浅入深 · 问题 1—2

Q01 Pi 到底是应用还是库?

【源码证实】完整终端应用是 @earendil-works/pi-coding-agent；它声明 bin.pi。pi-ai 提供模型适配，pi-agent-core 管循环/状态，pi-tui 管界面。单独引用库可复用能力，但不会自动得到完整产品。用户自述的 0.84.2 是该产品包版本，不是另一个“底层库版本”。



箭头表示职责拆解，不是所有直接依赖边

packages/coding-agent/package.json:1-L24 · b30d54b8；packages/coding-agent/package.json:1-L29 · 914cf147

Q02 为什么旧版没有教程里的 MCP?

【源码证实】0.84.2 README 明写 No MCP，建议外接扩展；真实发布物的依赖没有 pi-mcp 与 pi-codemode。本次运行断言确认缺席的是内置实现，不代表第三方扩展不可能存在。0.99.0 的公开变更记录才加入这套内置 MCP/codemode，不能对 0.84.2 直接套用 pi mcp add/list。



没有新命令，不等于旧版本安装坏了

packages/coding-agent/README.md:494-L506 · 914cf147；packages/coding-agent/CHANGELOG.md:678-L709 · 65c43bb5

由浅入深 · 问题 3—4

Q03 原帖的“11 个包”说错了吗?

【作者说法】原帖说 11 个主包、10 个公开包。【源码证实＋我实际运行过（云端隔离环境）】与 1.2.0 的 package.json 快照符合；本次 main 是 12/11，新增 schema。作者未给 SHA，所以只能说“数量吻合”，不能倒推作者实际 checkout 了 1.2.0。private 只是声明，本次未再逐包测试 npm 可安装性。

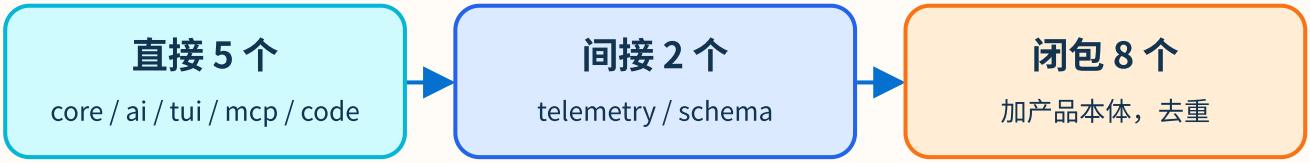


固定计数口径：packages/*/package.json

packages/schema/package.json:1-L60 · b30d54b8；packages/coding-agent/package.json:42-L67 · 06a46693

Q04 直接依赖和间接依赖怎么数?

【源码证实】main 产品直接依赖五个自家库；pi-ai 再依赖 telemetry 与 schema，agent-core 也依赖 ai/schema。去重后加本体，共八个。不要按树上重复出现的节点计数：ai 同时由产品与 agent-core 引入，仍只算一个包。闭包计算只跟随 dependencies，不混入开发测试工具。



依赖图不是“每层数量相加”的树

packages/ai/package.json:94-L119 · b30d54b8；packages/agent/package.json:1-L50 · b30d54b8

由浅入深 · 问题 5—6

Q05 chord / durable / env 与 CLI 无关吗?

【源码证实＋我实际运行过（云端隔离环境）】1.2.0、旧审计与本次 main 的产品生产依赖闭包均没有这三个包；但 monorepo 仍保留它们。env 用 durable/chord，durable 用 ai/codemode，并不推出 CLI 必须用 durable。“不是当前依赖主链”比“永远没有关系”准确，历史 chord 依赖的移除正是反例。



这是支线的方向，不是 CLI 反向依赖它们

packages/env/package.json:1-L54 · b30d54b8；packages/durable/package.json:1-L174 · b30d54b8

Q06 不依赖官方 SDK 还能算 MCP 吗?

【源码证实】当前 pi-mcp 生产外部依赖仅 cross-spawn；仓库自带 client/protocol/transports/OAuth 源码。是否使用 @modelcontextprotocol/sdk 和是否实现协议不是同一判断。当前配置支持 stdio 与 Streamable HTTP，显式拒绝旧 SSE 配置。【未验证】本次没有启动 MCP server 或做协议互操作测试。



遵守协议不要求依赖某一个 SDK

packages/mcp/package.json:1-L60 · b30d54b8；packages/mcp/src/config.ts:85-L140 · b30d54b8

由浅入深 · 问题 7—8

Q07 codemode 就是在本地跑 Node 吗?

【源码证实】不是；当前实现使用 QuickJS/WASI。脚本是 async 函数体，可 await/return，没有直接 Node API、文件系统、网络或 timer；外部效果通过工具与 models 暴露。代码在 CLI 所在环境运行，不证明模型推理在本地。【作者说法】作者回复称“本地在里面生成代码并执行”。【未验证】本次未运行 QuickJS，也未测试模型生成脚本。



脚本本地执行 ≠ 模型本地推理

packages/coding-agent/docs/codemode.md:3-L18 · b30d54b8；packages/codemode/src/runtime/vm.ts:160-L180 · b30d54b8

Q08 mode:on 是否表示默认开着工具?

【源码证实】当前内置扩展注册 defaultActive:false。codemode.mode 的 on 描述激活后的呈现模式，不等于启动即把工具加入活跃集合。可经 --tools、defaultTools 或 SDK 激活，MCP 的暴露策略也可能激活。这个结论仅限固定 1.5.2；0.84.2 没有后来这套内置 codemode，不能复制这些配置回旧版。



注册、激活、呈现是三层状态

packages/coding-agent/src/extensions/codemode/index.ts:1-L45 · b30d54b8；packages/coding-agent/src/core/settings-defaults.ts:57-L60 · b30d54b8

由浅入深 · 问题 9—10

Q09 “telemetry 库不上报”意味着零遥测?

【源码证实】不意味着。基础库的 NOOP/契约与 CLI 自己的安装/更新 ping 是两条路径。0.84.2 和当前版 install telemetry 默认 true，交互模式首次记录版本或更新版本时触发。本次 mock 实际捕获到 version 查询参数及含平台、运行时、架构的 UA，无 JSON body；没有真实发往 pi.dev，更没测服务端留存。



三个遥测/归因层，不能用一个开关替代全部

packages/coding-agent/src/core/settings-manager.ts:945-L955 · 914cf147；packages/coding-agent/src/modes/interactive/interactive-mode.ts:1192-L1239 · 914cf147

Q10 PI_TELEMETRY=0 能彻底断网吗?

【我实际运行过（云端隔离环境）】0.84.2 的真实判断函数十用例全部通过：环境变量优先，1/TRUE/yes 为真，0/false/no/空串/乱值为假；没有环境变量才读取设置。五分支 mock 显示 PI_OFFLINE 的非空字符串“0”也阻止该 ping。【源码证实】这都不是网络防火墙，不能证明显式模型请求、扩展或所有 provider 标识被禁。



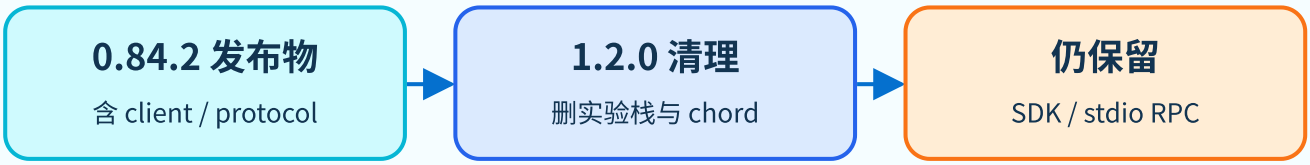
本次测的是函数与单方法，不是全进程抓包

packages/coding-agent/src/core/telemetry.ts:1-L13 · 914cf147；packages/coding-agent/src/core/provider-attribution.ts:36-L96 · 914cf147

由浅入深 · 问题 11—12

Q11 1.2.0 删 client，SDK / RPC 也没了吗?

【源码证实】删除的是当时实验 server/client/plugin 栈与 chord 产品依赖，本地 SDK 和 stdio RPC 仍在。0.84.2 真实 npm 发布物却有 ./client 导出和 pi-client/pi-protocol 依赖；不能把 1.2.0 说明中“未发布/source-only”的描述扩展成所有历史版本都没发布过 client。这是“发布说明”与“精确历史发布物”必须分开看的案例。



被删除的实验协议栈 ≠ 所有程序化入口

packages/coding-agent/CHANGELOG.md:61-L64 · 06a46693；packages/coding-agent/package.json:14-L29 · 914cf147；packages/coding-agent/README.md:460-L490 · 914cf147

Q12 有沙箱就可以放心加载扩展和 Skills 吗?

【源码证实】不能。Skill 是方法/指令资源，extension 是宿主可执行代码，SDK 是嵌入接口；工作目录不是文件访问边界。当前安全文档明确工具/扩展可用启动账户权限，项目 trust 不会把后续动作变安全。QuickJS 没有直接文件 API，也挡不住它调用的工具改文件或联网花钱。旧版 README 也说无权限弹窗，建议容器或自建确认流。【未验证】本次不装扩展、不运行付费能力。



审批该不该做，与沙箱能不能做，是两轴

packages/coding-agent/docs/security.md:1-L21 · b30d54b8；packages/coding-agent/README.md:494-L506 · 914cf147

用途：选最小的一层，而非全家桶

【源码证实】本次 main 的主包均为 1.5.2。下表只列自家生产依赖；完整外部依赖见旧审计与快照 package.json，未逐包运行集成。

主包	直接自家生产依赖	声明
chord	pi-schema, pi-telemetry	非 private
pi-agent-core	pi-ai, pi-schema	非 private
pi-ai	pi-schema, pi-telemetry	非 private
pi-codemode	—	非 private
pi-coding-agent	pi-agent-core, pi-ai, pi-codemode, pi-mcp, pi-tui	非 private
pi-durable	chord, pi-ai, pi-codemode, pi-schema, pi-telemetry	非 private
pi-env	chord, pi-durable	非 private
pi-evals	—	private
pi-mcp	—	非 private
pi-schema	—	非 private
pi-telemetry	—	非 private
pi-tui	—	非 private

五个可用方向（建议，非本次部署结果）

1. **诊断旧教程：**先认 0.84.2 的依赖/导出，再决定是否存在版本不匹配，而非立即升级。
2. **轻量嵌入：**只要模型适配时先评估 pi-ai；需要循环再加 agent-core；不是所有用途都要 CLI。
3. **扩展工作流：**代码扩展负责工具/事件，Skill 负责方法资源；兼容性以目标版本类型定义为准。
4. **构建审计素材：**把固定清单转依赖图，用源码链接为边作证；复用旧 vault 的“源码取证图”方法。
5. **隐私/升级检查：**分别审查安装 ping、provider 归因和基础 telemetry；不要把默认开关等同网络隔离。

主实验 · 六步中的 1—2

复现前提：Linux/macOS shell、Python 3、Node ≥22.19.0、npm、tar；先解压 exercise.zip 并 cd exercise。一次跑完：`python3 run.py`。离线复跑：归档自带 tgz，先在 lab 解压后运行第 2—6 步。所有脚本完整提供，不需任何密钥。

步骤 1 · 准备精确发布物

目的：下载精确公开发布物，避免误装当前版本；不安装依赖、不执行生命周期脚本。

```
cd lab
node --version && npm --version
npm pack @earendil-works/pi-coding-agent@0.84.2 --ignore-scripts --json > pack.json
tar -xzf earendil-works-pi-coding-agent-0.84.2.tgz
node -e "const p=require('./package/package.json');console.log(p.name+'@'+p.version)"
```

真实结果（本次运行，退出 0）：

```
v24.22.1
11.21.1
@earendil-works/pi-coding-agent@0.84.2
```

验证与边界：退出码 0；本体版本精确为 0.84.2；指纹在步骤 6 再验。

步骤 2 · 发布物身份与旧功能边界

目的：确认旧产品身份、五个直接自家依赖、client 导出与 No MCP 声明。

```
cd lab
node manifest.mjs
```

真实结果（本次运行，退出 0）：

```
@earendil-works/pi-coding-agent@0.84.2
own deps: @earendil-works/pi-agent-core,@earendil-works/pi-ai,@earendil-works/pi-client,@earendil-works/pi-protocol,@earendil-works/pi-tui
PASS: version/dependencies/exports; not a runtime MCP conformance test
```

验证与边界：全部断言退出 0；确认无 mcp/codemode，有 client/protocol；不排除用户自装扩展。

主实验 · 六步中的 3—4

复现前提：Linux/macOS shell、Python 3、Node ⩾22.19.0、npm、tar；先解压 exercise.zip 并 cd exercise。一次跑完：`python3 run.py`。
离线复跑：归档自带 tgz，先在 lab 解压后运行第 2—6 步。所有脚本完整提供，不需任何密钥。

步骤 3 · 四快照生产依赖闭包

目的：对四个 SHA 的主包声明计算去重生产依赖闭包；验证新旧图不串用。

```
python3 graph.py
```

真实结果（本次运行，退出 0）：

```
0.84.2: workspaces=9 non_private=8 closure=7
1.2.0: workspaces=11 non_private=10 closure=7
audit/main 1.5.2: workspaces=12 non_private=11 closure=8
PASS: 4 pinned snapshots; manifest hashes and production-closure assertions
（此处为真实输出摘要；完整 JSON 见 experiment-log.md）
```

验证与边界：四快照哈希检查通过；1.2.0 闭包 7、两份 1.5.2 闭包 8；非 runtime import 图。

步骤 4 · 真实开关函数十用例

目的：直接导入 0.84.2 编译纯函数，验证设置与环境变量的优先级。

```
cd lab
node switch.mjs
```

真实结果（本次运行，退出 0）：

```
env=<unset>, setting=true → true; setting=false → false
env=0/false/no/空串/garbage → false
env=1/TRUE/yes, setting=false → true
PASS: 10 telemetry-switch cases; no fetch or model invocation
（此处为真实输出摘要；完整 JSON 见 experiment-log.md）
```

验证与边界：10 个预期断言通过，无 fetch；SettingsManager 默认 true 仍来自静态源码。

主实验 · 六步中的 5—6

复现前提：Linux/macOS shell、Python 3、Node ⩾22.19.0、npm、tar；先解压 exercise.zip 并 cd exercise。一次跑完：`python3 run.py`。
离线复跑：归档自带 tgz，先在 lab 解压后运行第 2—6 步。所有脚本完整提供，不需任何密钥。

步骤 5 · 真实编译方法+mock请求五分支

目的：抽出真实编译安装 ping 方法，以 mock fetch 验证计划请求和阻断分支。

```
cd lab
node ping-mock.mjs
```

真实结果（本次运行，退出 0）：

```
{"url":"https://pi.dev/api/report-install?version=0.84.2","opts":{"headers":{"User-Agent":"pi/0.84.2
(linux; node/v24.22.1; x64)"},"signal":{"mockTimeoutMs":5000}}}
branch {"setting":true,"count":1} mock requests 1
branch {"flag":"0","setting":true,"count":0} mock requests 0
branch {"offline":"1","setting":true,"count":0} mock requests 0
branch {"offline":"0","setting":true,"count":0} mock requests 0
branch {"setting":false,"count":0} mock requests 0
PASS: 5 mocked branches; no real fetch or model invocation
```

验证与边界：5 个分支通过；启用 1 条、关闭 0 条、offline=1/0 各 0 条；不证明 UI 触发或真实服务器行为。

步骤 6 · 输入完整性及范围复核

目的：复验归档指纹与结果数量，明确本次组件层级与未测范围。

```
python3 final-check.py
```

真实结果（本次运行，退出 0）：

```
PASS: tarball SHA1 e4d4c1e769963c816959f5cea02a0a10ccc0495a; 4 snapshots present
LIMIT: no CLI session, provider, MCP interop, server privacy, or user machine tested
```

验证与边界：SHA1 与已知 registry shasum 相同；4 份图结果存在；没有主 CLI/模型/MCP 或用户机验收。

1.2.0 变更与证据边界

不要只记“删了 chord”

【源码证实】固定 1.2.0 CHANGELOG：配置 schema 发布与 theme 迁移；before_agent_start 支持扩展触发的 run 并改变事件类型；启动阶段视作 busy；MCP resources 改布尔标志、不再连接时列举所有资源；工具新增 reachesTools / namespace aliases；codemode 显示约束、耗时并修复 options 行检查；print 输出截断时非零退出；最后才是删除实验栈与 chord 依赖。

packages/coding-agent/CHANGELOG.md:3-L64 · 06a46693

静态变更记录 ≠ 本次运行验证

以上是源码/发布说明记录，不是本次复现所有 bug fix。旧报告核查 GitHub release 的上海发布时间为 2026-10-13 07:03:53；CHANGELOG 标题为 10 月 12 日，两者不可混同。

哪些东西没有验证?

- 【未验证】用户本机是否仍安装 0.84.2、安装路径、扩展、配置与环境变量；没有访问本机。
- 【未验证】完整 CLI 的 --help/--version、交互首启、全进程抓包与真实 pi.dev 服务端留存。没有把 PI_OFFLINE 宣传为 OS 防火墙。
- 【未验证】任何真实模型、计费、SDK prompt、MCP 连接/OAuth/互操作、QuickJS 端到端执行与安全逃逸。
- 【未验证】作者提及的 Star、OpenClaw 采用历史、提交频率、订阅适用范围与性能收益。不把作者数字当实测。

这次实际执行了什么?

【我实际运行过（云端隔离环境）】公开下载 npm 发布包、只读清单与源码、直接运行纯函数、在 VM 内给真实单方法注入 mock；六步退出码全为 0。mock 环境没有真实 fetch，不会向遥测站点发请求。它验证分支逻辑，不验证真实网络隐私。

研究准备时一次调用 python（本机命令仅为 python3）失败，退出 127；改用 python3 后成功。X 作者线程第二页曾遇共享 HTTP 429；保留原因，不虚构返回内容。详细情况见 sources.md。

查重与来源：只新增，不覆盖

【我实际运行过（云端隔离环境）】公开 reminder_knowlege 当前 main 固定为 `a8674777e18302b8804b137212b1396cbfe5a747`。已读 vault/00-总览.md 全文、项目笔记/agent-kernel.md、项目笔记/archify.md、源码取证图、事实与判断分离、固定版本中的跨文件时间漂移，并读 archify HTML 的版式定义。检索未找到这条 Pi 架构原帖的独立项目笔记；但通用 Agent 循环、MCP、Skills、沙箱/审批已覆盖，不重复新建同义概念。

给总协调者的增量建议

- 新增候选：vault/项目笔记/pi-architecture.md，存版本边界、四快照图及六步实际结果。
- 增补“源码取证图”：生产依赖闭包与 runtime import 图不能互换；7 包同数量可能完全不同组成。
- 增补“固定版本中的跨文件时间漂移”：旧 HEAD 不能永久称当前；发布说明也不能擦除历史 npm client 导出。
- 增补“沙箱与审批正交 / 事实与判断分离”：真实函数+mock 单方法通过，不替代真实 CLI、权限/计费或服务端隐私验收。

核心一手证据（完整 URL 与读取状态在 sources.md）

1. 当前产品声明 `b30d54b8`
2. 旧版产品声明 `914cf147`
3. 旧版 No MCP/权限哲学 `914cf147`
4. 旧版遥测开关 `914cf147`
5. 旧版安装 ping `914cf147`
6. 1.2.0 变更 `06a46693`
7. 当前安全边界 `b30d54b8`
8. 当前 codemode `b30d54b8`

旧审计的 93 处行号引用保留在 evidence/prior-audit.md；本次不是把其“未执行实验”改写成已执行，而是另用 experiment-log.md 为这次新运行单独记账。图仅为教学简图，准确边以固定 package.json 和 graph-results.json 为准。

> 本项交付完成不代表用户任务已被勾选；知识合入、领取、网站标签、Git 发布由总协调者管理。本 worker 只在自己的候选目录保存产物。